BookLeaf Portal - Approach and Trade-offs

Live demo: https://authors-support-portal.vercel.app/

I prioritised reliable author support, strict ownership boundaries and human control of AI responses. The portal uses a Next.js modular monolith with separate HTTP, authentication, business-service, repository and AI boundaries. Supabase provides email/password identity and PostgreSQL; Prisma manages the application schema. A single deployment keeps operations simple, while these boundaries make business rules testable. Hosted-only local integration reduces infrastructure setup but requires network access and a configured Supabase project.

The source import preserves ten authors and eighteen books, including production nulls and platform arrays. Application users share Supabase Auth UUIDs; profiles retain original business IDs and contacts. The approved isolated demo uses exact source emails and two admins, with privileged setup gated by project approval and outbound-mail blocking. A composite foreign key protects ticket/book ownership; messages, private notes and events have separate tables.

The implementation includes HTTP-only authentication, scoped books, provisioning, ticket conversations and admin lifecycle/private history. Queue date filters and oldest-unresolved controls support triage. Validation, retry IDs and shared locks protect ownership, closing and replies. AI classification runs after persistence with a deadline and cannot overwrite manual decisions. Drafts use selected fixed policies and bounded public context, with a persisted fingerprint cache. Prompts prohibit unsupported operational claims; admins review, edit and explicitly send. AI failure leaves manual support available.

I chose polling and a small versioned Knowledge Base over real-time infrastructure and vector retrieval: both fit the current scale, with some refresh latency and limited policy coverage. Bounded synchronous AI calls simplify deployment but add request latency; persisted drafts and output caps reduce repeat cost. Attachments and ticket links are deferred to prioritise the core support workflow. For production, I would add usage quotas and budget enforcement, operational monitoring, broader security/load testing and live-model evaluations. Durable background jobs would handle AI retries as demand grows; private Cloudflare R2 storage would support attachments.

Tests cover policies, handlers, retries, lifecycle, source integrity and AI failure/cache behavior. Hosted safety, admin/AI workflows, browser flows and a real synthetic OpenAI smoke passed. Exact verification scope and limits are documented in TESTING.md; larger-scale operations and broad adversarial model evaluation remain follow-up work.